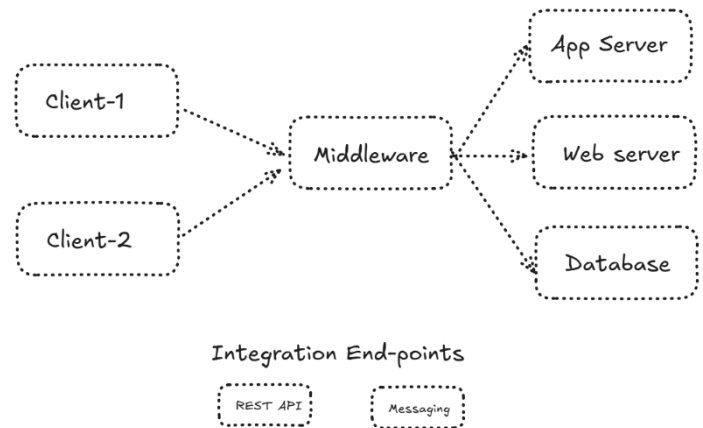


Middleware

- Middleware is the layer between the apps or OS, enabling them to communicate and exchange data smoothly.
- Different types of middleware:
 - ✓ **Message oriented** – it queues the data (messages). Ex: RabbitMQ and Kafka.
 - ✓ **Object request brokers** – remote method calls. Ex: Cobra.
 - ✓ **Database middleware** – connects applications to database. Ex: JDBC & ODBC
 - ✓ **RPC** – invokes functions on a remote system. Ex: gRPC and SOAP.
 - ✓ **Web middleware** – hosts and routes web traffic. Ex: Apache, tomcat, Nginx
 - ✓ **API gateway** – manages and secures API traffic. Ex: Apigee.
- Where middleware helps you in the enterprise systems:
 - ✓ It connects various services together (web, db).
 - ✓ It allows the integration across hybrid cloud platform.
 - ✓ It helps in microservices.



Introduction to web services:

- ✓ Web services are the standardized way of integrating web-based applications using protocols and data formats like HTTP, XML, JSON, SOAP and REST APIs.
- ✓ These web services allow an application to communicate and share data and services among themselves over internet.
- ✓ It acts as a bridge between apps or OS or any platform.

Why to use web services?

- ✓ Web services provide interoperability – they allow different apps written in different languages and running on different platforms to communicate.
- ✓ Benefits of using web services:
 - Language and platform independent.
 - Scalable
 - Firewall friendly (uses the standard ports 80/443)
- ✓ Disadvantages:
 - Latency (for remote communication).
 - Security risks, if not properly secured.

Types of web services:

SOAP

- SOAP is a protocol-based web service which uses XML to encode your requests and responses to run over HTTP/HTTPS.
- It's a protocol with strict standards.
- Platform and language independent.
- It is used within the enterprise-level apps like banking and financial transactions.

REST web services

- REST uses HTTP methods (GET, POST, PUT & so on.)
- It is much light-weight and fast.
- It uses JSON or XML.
- Easily consumed by mobile, web and cloud apps.
- Ex: X.com, Google Maps.

XML-RPC / JSON-RPC

- These protocols use XML/JSON to encode data and send it via HTTP.
- Simpler than SOAP and much more light-weight.

gRPC

- Google Remote procedure call.
- It is modern high-performance RPC developed by Google.
- Useful in Microservices and Kubernetes.

OData Protocol

- Open Data Protocol
- This is REST-based protocol for querying and updating environments.

WebSocket

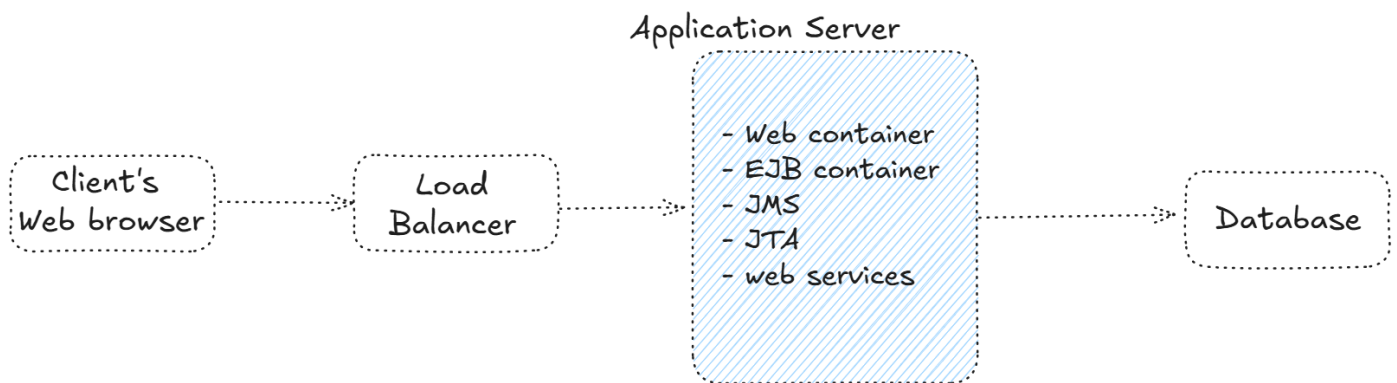
- It allows the bi-directional TCP connection.
- This is used in real-time web services like chats, IoT.

Application server and Integration endpoints:

Application server

- An application server is a software framework that provides enterprise applications to run smoothly, securely and efficiently.
- It acts as a bridge between the front-end client/apps and back-end database.
- We can deploy this is business logic, webservice or middleware components.
- Why to choose application server:
 - ✓ Hosting web application or enterprise applications.
 - ✓ Managing the session state and resources.
 - ✓ For performing load balancing, clustering and failover.
- Examples:
 - ✓ Apache tomcat → Java-based
 - ✓ JBoss → Java-based
 - ✓ Weblogic → Java-based
 - ✓ IBM web sphere → Java-based
 - ✓ Microsoft IIS → .NET-based
 - ✓ Nginx/Apache

Architecture of application server:



Application server and Integration endpoints:

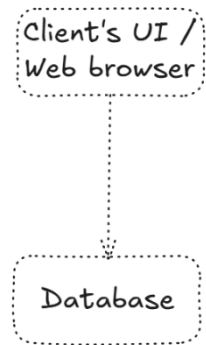
Integration endpoints

- Integration endpoints are the interfaces where exchange of data or services happens.
- Basically, it's the entry or exit point for communication between systems.
- Types of integration services:
 - ✓ Web service endpoint
 - ✓ Database endpoint
 - ✓ Message queue endpoint
 - ✓ FTP endpoint
 - ✓ API Gateway endpoint
 - ✓ Cloud service endpoint

Two-tier and Three-tier architecture

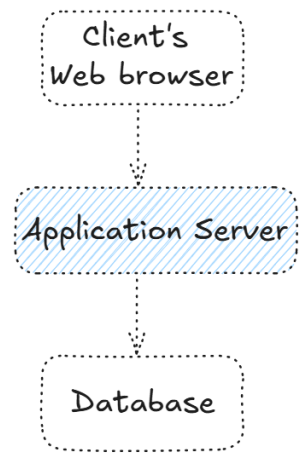
Two-tier architecture

- It is a client-server architecture where the client communicates with backend (database) directly.
- Layers:
 - ✓ Client tier → UI / Application / browser
 - ✓ Database tier → MySQL, Microsoft SQL Server, Oracle.
- Simple and faster.
- We use it for the small-scale application due to its easier development and maintenance.



Three-tier (n-tier) architecture

- This architecture separates an application into the presentation, business logic and database layer.
- This makes it more modular and scalable.
- Layers in 3-tier architecture:
 - ✓ Presentation tier – frontend (web/mobile)
 - ✓ Application tier – middleware or application server
 - ✓ Database tier – database server
- Highly scalable
- Better security than 2-tier
- Easy to maintain
- Design is complex / complex design.



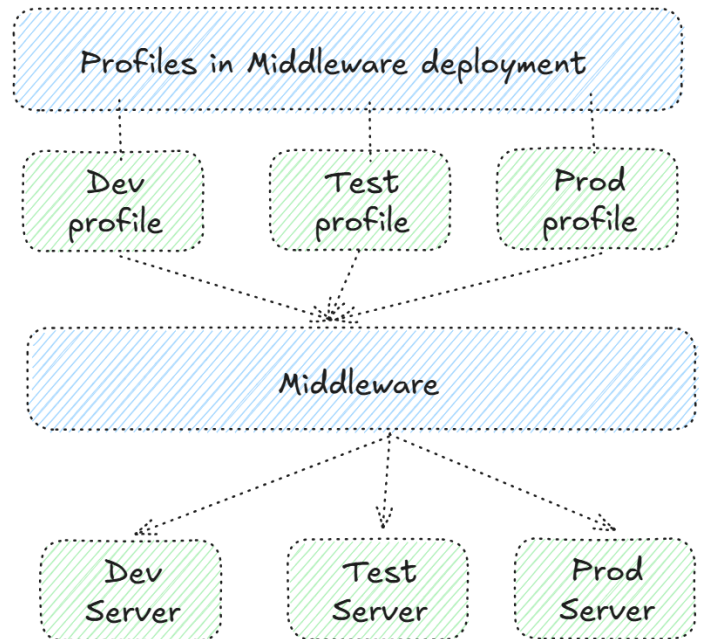
Install an Application Server

For windows / Linux

- Prerequisites
 - ✓ JDK 8 or above must be installed.
 - ✓ Sufficient space (disk) and memory (RAM).
 - ✓ Must have admin (root / administrator) privileges.
 - ✓ Firewall → 8080, 8443, 80 & 443.
- Download the server application
 - ✓ Tomcat → <https://tomcat.apache.org/download-90.cgi>
 - ✓ Web logic → <https://www.oracle.com/middleware/technologies/weblogic-server-installers-downloads.html>
- Configure the environment variables
 - ✓ Set JAVA_HOME variable.
- Start the server and access it
 - ✓ URL: <http://localhost:8080>

Introduction to Profiles

- ✓ Profile is commonly used in application servers and integration endpoint platforms.
- ✓ In middleware, it refers to a customizable configuration set that determines how a middleware component behaves or runs.
- ✓ Behavior like:
 - Startup mode
 - Enables services
 - Memory and JVM settings
 - Security
 - Logs
 - Runtime roles (admin, client)
- ✓ Purpose of using profile:
 - Configuration management (CM)
 - Feature isolation
 - Flexible deployment
 - Easy packaging



Examples of Profiles in middleware technologies:

1. WebSphere Application Server (WAS)
 - Default profile → standard server setup.
 - Admin profile → central domain management
 - Application server profile → base profile to host application
 - Custom profile → used for federation nodes
2. Tomcat
 - Separate configuration folders for different profiles.
 - Startup scripts with JAVA_OPTS
 - Environment-specific WAR deployments

Federation profiles:

- ✓ A federation profile defines how identities, tokens and trust relationship are established and managed across domains.
- ✓ A federation profile is a configuration template that includes:
 - IdP (Identity provider)
 - Service provider
 - Protocol
 - Certificates
 - User attributes
- ✓ Common federation profiles:
 - SAML web SSO
 - OAuth 2.0
 - WS-Federation
- ✓ Tools that use federation profile:
 - Azure AD → Microsoft EntraID B2C
 - Okta
 - Auth0
 - PingFederate

Installing Apache Tomcat on CentOS 9

Prerequisites

- CentOS 9 Stream or CentOS 9 minimal or any linux OS
- Sudo privileges → login with “root” user.
- Internet access

Installing Java

- ✓ Tomcat needs Java
- ✓ Apache Tomcat requires Java 8 or above.
- ✓ You have to install OpenJDK
 - # `yum install java-17-openjdk -y`

Commands to execute:

1. Ping to google.com to check internet connectivity:

```
# ping google.com
```

2. Install the Java package:

```
# yum install java-17-openjdk -y           //or
# dnf install java-17-openjdk -y
# yum install -y java-17-openjdk-devel java-11-openjdk-devel
```

3. Create a new user (tomcat)

```
# useradd -r -m -U -d /opt/tomcat -s /bin/false tomcat
-r → creating a system user
-m → create the home directory of the user.
-U → create a user group (tomcat)
-d /opt/tomcat → sets home directory
-s /bin/false → non-login shell
-tomcat → username to be created
```

4. Verify, if user is created

```
# cat /etc/passwd | grep tomcat
```

5. Change the location to /tmp

```
# cd /tmp
```

6. Download the Apache tomcat linux package using “wget” command

```
# wget https://downloads.apache.org/tomcat/tomcat-10/v10.1.43/bin/apache-tomcat-10.1.43.tar.gz
```

7. Extract this ‘gz’ file and move the extracted data (files and folder) to /opt/tomcat

```
# tar -xzf apache-tomcat-10.1.43.tar.gz -C /opt/tomcat --strip-components=1
```

8. Change the permissions of newly copied files

```
# chown -R tomcat: /opt/tomcat
```

9. Make all scripts executable

```
# sh -c 'chmod +x /opt/tomcat/bin/*.sh'
```

10. Create a systemd service file.

vim /etc/systemd/system/tomcat.service & paste the below code inside this file

```
[Unit]
Description=Apache Tomcat Web Application Container
After=network.target

[Service]
Type=forking

User=tomcat
Group=tomcat

Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk"
Environment="CATALINA_PID=/opt/tomcat/temp/tomcat.pid"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"

ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

Restart=on-failure

[Install]
WantedBy=multi-user.target
```

:wq!

11. Reload daemon (service) and start Tomcat

```
# systemctl daemon-reexec
# systemctl daemon-reload
# systemctl enable --now tomcat
```

12. Check the tomcat service status.

```
# systemctl status tomcat
```

13. If firewall is ON

```
# sudo firewall-cmd --permanent --add-port=8080/tcp
# sudo firewall-cmd --reload
```

14. Test the Apache tomcat page on the browser.

Go to firefox web browser → <http://<ip-address>:8080>

15. Enable Tomcat web management interface → Edit a file

```
# vim /opt/tomcat/conf/tomcat-users.xml
<role rolename="manager-gui"/>
<role rolename="admin-gui"/>
<user username="admin" password="admin123" roles="manager-gui,admin-gui"/>
```

:wq!

16. Allow the remote access by removing/commenting line from the following files.

```
# vim /opt/tomcat/webapps/manager/META-INF/context.xml
# vim /opt/tomcat/webapps/host-manager/META-INF/context.xml
```

Comment using <!--

```
<Valve className="org.apache.catalina.valves.RemoteAddrValve"
    allow="127\.\d+\.\d+\.\d+|::1"/>
```

-->

:wq!

17. Now restart the tomcat service

```
# systemctl restart tomcat
# systemctl status tomcat
```

18. Go to browser, refresh it again.

19. Type username as "admin" and password as "admin123".

20. Click on "Server Status" button.